

Comprehensive Big Data and Hadoop Ecosystem Notes

Generated Document

Contents

1	Big Data Overview	2
1.1	What is Big Data?	2
1.2	The 5 V's of Big Data	2
2	Hadoop Ecosystem	2
2.1	Features of Hadoop	2
2.2	Ecosystem Components	3
3	HDFS (Hadoop Distributed File System)	3
3.1	Architecture Components	3
3.2	Rack Awareness & Operations	3
4	MapReduce and YARN	4
4.1	MapReduce	4
4.2	YARN (Yet Another Resource Negotiator)	4
5	Apache Hive	4
5.1	Architecture	5
5.2	Hive Execution Flow	5
5.3	Tables, Partitions, and Buckets	5
6	Apache HBase	5
6.1	HBase vs RDBMS & HDFS	5
6.2	Data Model and Architecture	6
7	Apache Kafka	6
7.1	Core Concepts	6
7.2	Core APIs	6
8	Apache Spark	7
8.1	Spark Components	7
8.2	Resilient Distributed Datasets (RDD)	7
8.3	DataFrame and DataSet	7

1 Big Data Overview

1.1 What is Big Data?

Big Data refers to a collection of data that is huge in volume (in petabytes) and growing exponentially. Traditional databases (RDBMS) cannot handle and process this large amount of data efficiently.

- **Big Data vs RDBMS:** RDBMS uses a fixed schema (static) and works only with structured data. It performs historical data analytics and has a centralized architecture. Big Data handles structured, semi-structured, and unstructured data (dynamic nature). It scales on-the-fly and supports real-time analytics with a distributed architecture.
- **Benefits:** Better decision making, greater innovations, product price optimization, and life-saving applications (e.g., healthcare).
- **Challenges:** Sharing, transferring, and analyzing immense data volumes.

1.2 The 5 V's of Big Data

- **Volume:** Enormous amount of data (e.g., petabytes of social media data).
- **Velocity:** Rate of data generation (e.g., Google searches, Twitter feeds).
- **Variety:** Different types of data (structured, semi-structured, unstructured).
- **Veracity:** Inconsistencies and uncertainty in data (messiness, quality, accuracy).
- **Value:** The useful insights that the data can provide.

2 Hadoop Ecosystem

Hadoop is an open-source software framework used to develop data processing applications that execute in a distributed computing environment across clusters of commodity computers.

2.1 Features of Hadoop

- **Reliability:** Node failures do not disable the cluster; another node takes over.
- **Scalable:** Nodes can be added on-the-fly.
- **Economical:** Uses cheap commodity hardware.
- **Distributed Processing & Storage:** Jobs are divided into independent sub-tasks, and files are split into blocks stored across multiple machines.
- **Fault-Tolerance & High Availability:** Achieved through block replication and standby NameNodes.
- **Data Locality:** Computation logic is moved to the data, reducing network bandwidth usage.

2.2 Ecosystem Components

- **HDFS:** Distributed storage.
- **MapReduce & YARN:** Data processing and cluster resource management.
- **Hive:** Data warehouse project for query and analysis (SQL-like).
- **Pig:** Scripting language for analysis.
- **HBase:** NoSQL columnar database on top of HDFS.
- **Mahout:** Scalable Machine Learning library.
- **Zookeeper:** Coordination service.
- **Oozie:** Workflow scheduler.
- **Sqoop & Flume:** Data ingestion tools for structured and unstructured data, respectively.
- **Spark:** Unified big data processing framework (in-memory).

3 HDFS (Hadoop Distributed File System)

HDFS uses a master-slave architecture. Files are initially divided into blocks (default size 128MB) and saved on DataNodes depending on the replication factor.

3.1 Architecture Components

- **NameNode (Master):** Maintains and manages the file system namespace. Keeps metadata (block locations, permissions). Metadata is stored in RAM for fast access and persisted in the **FsImage** file. Changes are logged in the **EditLog**.
- **DataNode (Slave):** Stores actual data blocks. Serves client read/write requests. Sends heartbeats and block reports to the NameNode.
- **Secondary NameNode:** Periodically merges **FsImage** and **EditLog** to prevent the **EditLog** from becoming too large, reducing NameNode restart time.
- **Backup/Checkpoint Node:** Keeps an up-to-date in-memory copy of the file system namespace and synchronizes with the active NameNode.

3.2 Rack Awareness & Operations

- **Rack Awareness:** The concept of choosing the closest node based on rack information to reduce latency and provide fault tolerance.
- **Write Operation:** Client asks NameNode for metadata → NameNode provides block locations → Client writes block to DataNode 1 → DataNode 1 pipelines to DataNode 2 (same rack) → DataNode 2 pipelines to DataNode 3 (different rack).

- **Read Operation:** Client asks NameNode for block locations → Client reads blocks in parallel from DataNodes → Client combines blocks into the original file.
- **High Availability:** Multiple NameNodes (Active and Standby) synchronized via JournalNodes. Fencing prevents split-brain scenarios.

4 MapReduce and YARN

4.1 MapReduce

MapReduce is the processing layer in Hadoop where large datasets are processed in parallel independent tasks.

- **Terminologies:** Job (full execution), Task (execution on a data slice), Task Attempt.
- **Map Abstraction:** Takes key-value pairs as input, applies custom logic, and outputs intermediate key-value pairs.
- **Reduce Abstraction:** Takes intermediate key-value pairs, performs aggregation/sorting, and writes the final output to HDFS.
- **Execution Flow:** Input Files → InputFormat → Input Splits → Record Reader → Mapper → Combiner (mini-reducer for local aggregation) → Partitioner → Shuffle & Sort → Reducer → Record Writer → OutputFormat.
- **Input Formats:** TextInputFormat (default), KeyValueTextInputFormat, SequenceFileInputFormat, DBInputFormat.

4.2 YARN (Yet Another Resource Negotiator)

Introduced in Hadoop 2, YARN is the cluster resource management layer.

- **Resource Manager (RM):** Master daemon responsible for resource assignment. Contains a Scheduler and an Applications Manager.
- **Node Manager (NM):** Manages applications and workflows on a particular node. Sends heartbeats to RM.
- **Application Master (AM):** Negotiates resource containers from the Scheduler and tracks container status.
- **Container:** Incorporates resources like CPU, memory, disk, and network.

5 Apache Hive

Hive is an open-source data warehouse system built on top of Hadoop for querying and analyzing large datasets using a SQL-like language (HQL). It converts queries into a series of jobs (MapReduce/Tez) for execution.

5.1 Architecture

- **Clients:** Thrift, JDBC, ODBC.
- **Services:** Beeline, HiveServer2, Driver, Compiler, Optimizer, Execution Engine.
- **Metastore:** Central repository storing metadata about table structures and partitions. It can run in Remote, Local, or Embedded mode.

5.2 Hive Execution Flow

1. User executes a query.
2. Driver creates a session and requests an execution plan from the Compiler.
3. Compiler fetches metadata from the Metastore to perform type-checking and semantic analysis.
4. Compiler generates a Directed Acyclic Graph (DAG) execution plan.
5. Optimizer improves the plan.
6. Execution Engine processes the tasks via MapReduce/YARN.

5.3 Tables, Partitions, and Buckets

- **Managed vs External Tables:** Dropping a Managed table deletes both schema and data. Dropping an External table deletes only the schema; the data remains in HDFS.
- **Partitions:** Divides a table into related parts based on column values (e.g., date, city). Can be Static (manual) or Dynamic.
- **Buckets:** Segregates data into smaller, manageable files based on a hashing function on a column value, improving query efficiency.
- **Columnar Storage:** ORC (Optimized Row Columnar) and Parquet save disk space and reduce I/O.

6 Apache HBase

HBase is an open-source, horizontally scalable, distributed column-oriented NoSQL database built on top of HDFS, providing random, real-time read/write access.

6.1 HBase vs RDBMS & HDFS

- **vs RDBMS:** HBase has no fixed schema, is scalable, handles sparse data, and favors the CAP theorem over ACID properties.
- **vs HDFS:** HDFS is for high-latency batch processing sequential access; HBase provides low-latency fast lookups for single rows.

6.2 Data Model and Architecture

- **Data Model:** Tables consist of rows. Rows contain Column Families. Column Families contain Columns, structured as Key-Value pairs.
- **HMaster:** Manages Region Servers, handles load balancing, and executes DDL operations.
- **Region Server:** Hosts regions, handles read/write requests, and manages Region splitting.
- **Zookeeper:** Maintains server state, configuration, and coordination.
- **Components:** Write-Ahead Log (WAL) for recovery, Block Cache (read cache), MemStore (write cache, sorted), and HFiles (permanent storage).
- **Compaction:** Minor (combines smaller files) and Major (combines files for the same column family to optimize reads).

7 Apache Kafka

Kafka is an open-source stream-processing platform and a publish-subscribe messaging system that handles real-time data storage and exchange between applications.

7.1 Core Concepts

- **Producer:** Sends/writes messages.
- **Consumer:** Reads/subscribes to messages.
- **Topics:** Categories or names used to store and publish data streams.
- **Partitions:** Topics are split into partitions. Messages are stored sequentially with an incremental ID called an *offset*.
- **Broker:** A server in the Kafka cluster holding multiple topics and partitions. Uses replication factors for fault tolerance.
- **Zookeeper:** Manages brokers, chooses leaders, and tracks consumer clients.

7.2 Core APIs

- **Producer API:** Publishes data to topics.
- **Consumer API:** Subscribes to topics.
- **Streams API:** Transforms input streams to output streams.
- **Connector API:** Integrates Kafka with existing data systems.

8 Apache Spark

Spark is an independent, low-latency, lightning-fast cluster computing framework that uses in-memory storage to process data.

8.1 Spark Components

- **Spark Core:** Foundation for parallel and distributed processing (provides RDDs).
- **Spark SQL:** Handles structured and semi-structured data.
- **Spark Streaming:** Scalable, high-throughput processing of live data streams using micro-batching.
- **MLlib:** Scalable Machine Learning library.
- **GraphX:** API for graphs and graph-parallel execution.

8.2 Resilient Distributed Datasets (RDD)

- **Features:** Immutability, distributed nature, fault tolerance (via lineage graphs), lazy evaluation, and in-memory computation.
- **Transformations:** Lazy operations creating a new RDD. Can be Narrow (e.g., `map`, `filter`) or Wide (e.g., `groupByKey`, `reduceByKey`).
- **Actions:** Trigger actual computation and return results to the driver program (e.g., `count`, `collect`).
- **Persistence/Caching:** Caching (`MEMORY_ONLY`) or persisting RDDs speeds up future computations.
- **Limitations:** No built-in optimization engine, manual schema handling, and potential garbage collection overheads.

8.3 DataFrame and DataSet

- **DataFrame:** Data organized into named columns (like an RDBMS table). Uses the Catalyst query optimizer.
- **DataSet:** Strongly typed, mapping to a relational schema. Provides compile-time safety and object-oriented interfaces. Faster computation and less memory consumption than raw RDDs.